

Chapitre 3

Tutoriel

Bienvenue dans votre premier tutoriel de programmation en **C** ! Avant de plonger dans le code, vous devez comprendre les bases. Regardez la vidéo suivante sur les langages de programmation et la mémoire de l'ordinateur : [Le langage de programmation C](#). Faites attention à la manière dont les ordinateurs stockent et manipulent les données.

- Comprendre la **mémoire** et le fonctionnement des programmes est crucial avant d'écrire votre première ligne de code **C**.
- Après avoir regardé la vidéo, créez votre premier programme C. En suivant l'exemple de la vidéo, créez une fonction `main` vide dans un fichier appelé `first_program.c` :

first_program.c

```
int main()
{
    return (0);
}
```

- Compilez-le en utilisant la commande suivante : `cc first_program.c -o first_program`
- Exécutez-le avec la commande `./first_program`. Que se passe-t-il ?
- Rien de visible ne se passe ? C'est normal. Vous n'avez pas encore d'instructions ! Modifiez votre code et ajoutez une instruction simple comme `1+1` ; à l'intérieur de la fonction `main`. Compilez et exécutez-le à nouveau.
- Toujours rien de visible ? En fait, votre code a fait quelque chose, mais vous ne le "voyez" tout simplement pas. L'ordinateur a calculé `1+1=2`, mais n'a rien affiché. Il est temps de rendre quelque chose visible !
- Vous devez afficher quelque chose dans le terminal. Créez la fonction `ft_putchar` qui affiche un caractère :

first_program.c

```
#include <unistd.h>

void ft_putchar(char c)
{
    write(1, &c, 1);
}
```

- Maintenant, créez un programme complet qui affiche un caractère. Appelez `ft_putchar` depuis votre fonction `main`. Le paramètre `char` est un code ASCII (un nombre entre 0 et 127 qui représente un caractère).
- Essayez d'afficher différents caractères :

first_program.c

```
ft_putchar('A');
ft_putchar('B');
ft_putchar('*');
```

Que se passe-t-il ?



Les codes ASCII sont la manière dont les ordinateurs représentent les caractères sous forme de nombres. 'A' est 65, 'a' est 97, '0' est 48, etc. Avec l'aide de vos pairs, essayez de trouver la commande shell qui affiche la table ASCII dans votre terminal.

- Maintenant, utilisons une variable ! Définissez une variable `char`, attribuez-lui une valeur de caractère ASCII, et affichez-la en utilisant `ft_putchar` :

first_program.c

```
char my_char;
my_char = 'X';
ft_putchar(my_char);
```

- Essayez d'ajouter un `\n` à la variable lors de l'appel de `ft_putchar` :

first_program.c

```
char my_char;  
my_char = 'X';  
ft_putchar(my_char + 1);
```

Quel caractère apparaît et pourquoi ?

- Maintenant, vous avez besoin d'une boucle ! Comment pourriez-vous répéter 10 fois l'affichage d'un caractère ? Recherchez les **variables de compteur** et la structure **while** en ligne. Essayez d'afficher le même caractère 10 fois de suite.
- Créez une boucle qui affiche différents caractères. Par exemple, affichez les lettres de A à J en utilisant une variable de compteur que vous ajoutez à une valeur de base ASCII.
- Les boucles sont fondamentales en programmation. Elles vous permettent de répéter des actions sans écrire le même code plusieurs fois.
- Expérimentez avec différents motifs de boucle : comptez à rebours, comptez à l'envers, affichez des motifs comme des étoiles ou des nombres...
- Vous êtes prêt pour les exercices ! Vous allez expérimenter avec diverses **boucles**, **branches conditionnelles** (if, else), et les mélanger ensemble. N'oubliez pas de demander de l'aide à vos pairs et de discuter de votre code.

La clé pour apprendre la programmation est la pratique, l'expérimentation et la discussion avec les pairs. N'hésitez pas à essayer différentes approches !